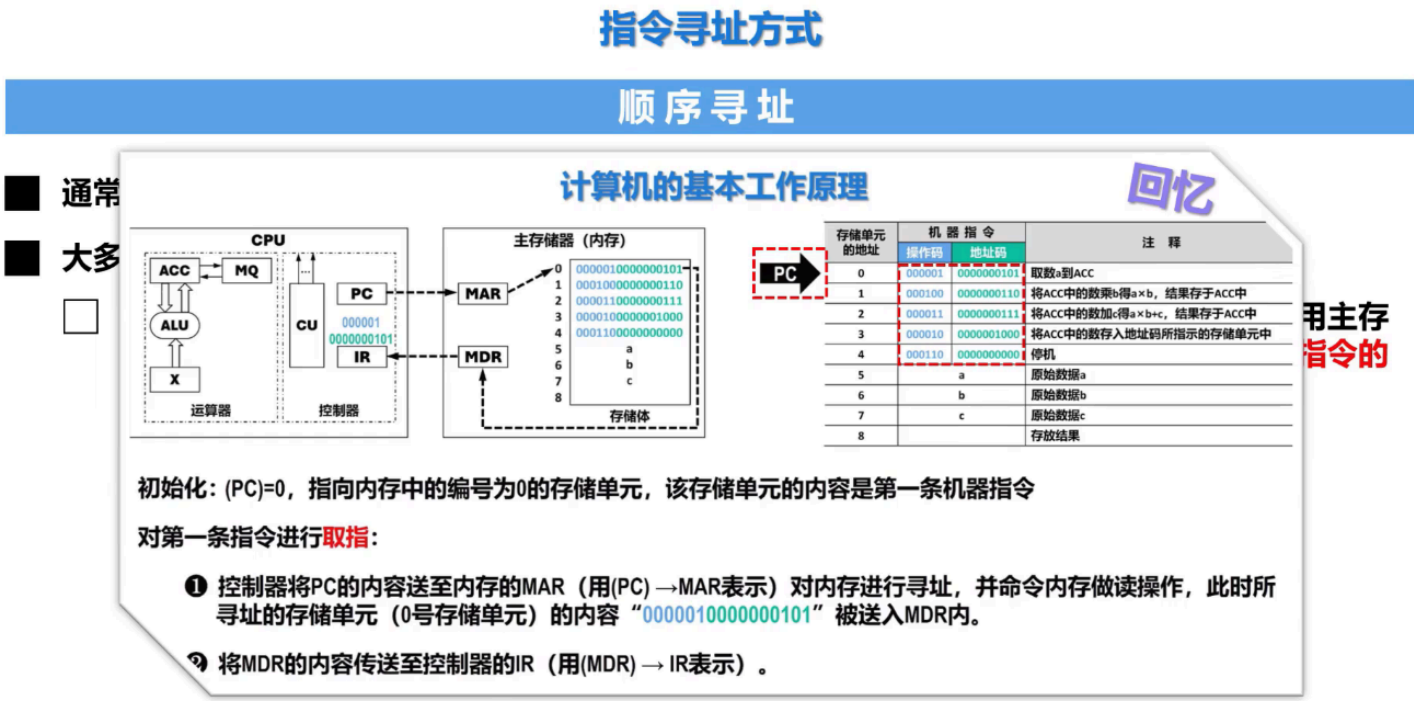
5.5 寻址方式——指令寻址

考纲定位：计算机组成原理 · 第 5 章 指令系统（5.5） | 重要度 ★★★★ | 常见考法：选择题（PC 增量、相对转移目标、指令字长与编址单位、取指与执行时序）

一句话核心：顺序寻址靠 PC 自增，跳跃寻址靠“已更新的 PC + 位移”或直接赋值

一、顺序寻址



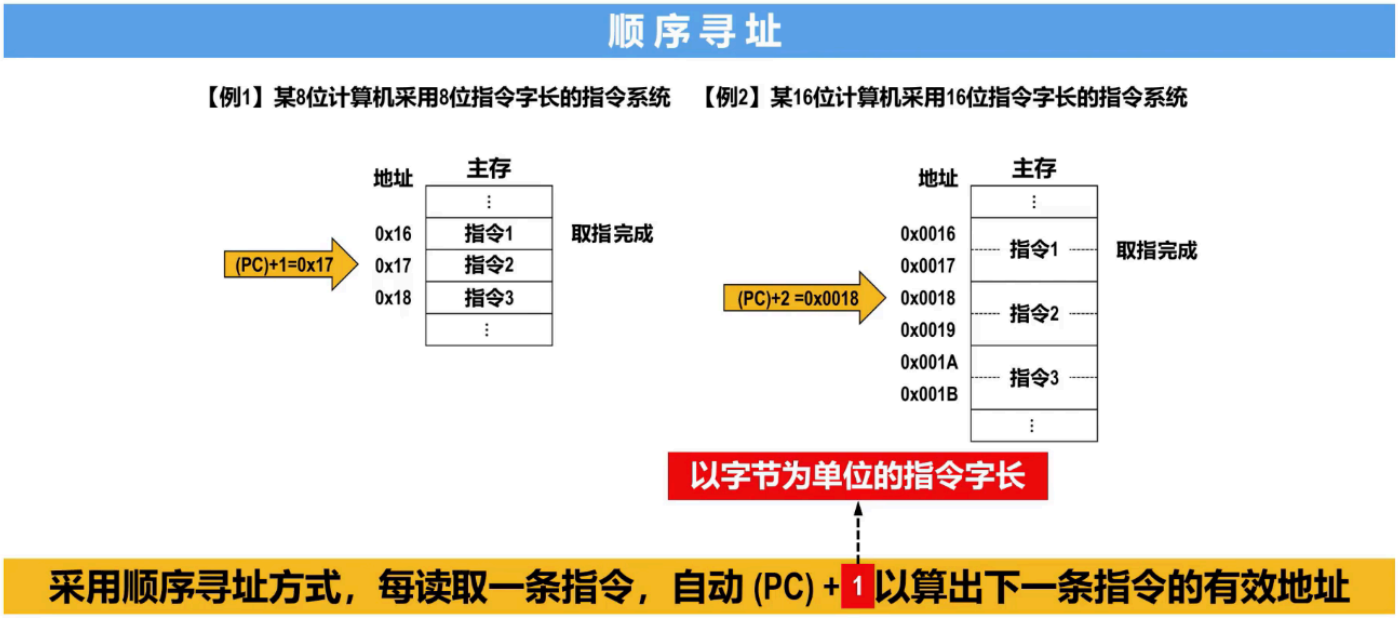
提炼

- 程序指令在主存中按顺序存放；取指后 PC 自动加上“本条指令占的字节数”
- PC 增量 = 指令字长 / 编址单位（单位统一；按字节编址时 = 指令字节数）
- 取指与 PC 更新通常并行：取指完成时 PC 已指向下一条指令

易错

- PC 增量不是固定 +1；只有“按字节编址且指令字长为 1 字节”时才是 +1
- PC 增量由“当前指令的长度”决定，不是由“下一条指令的长度”决定

举例



提炼

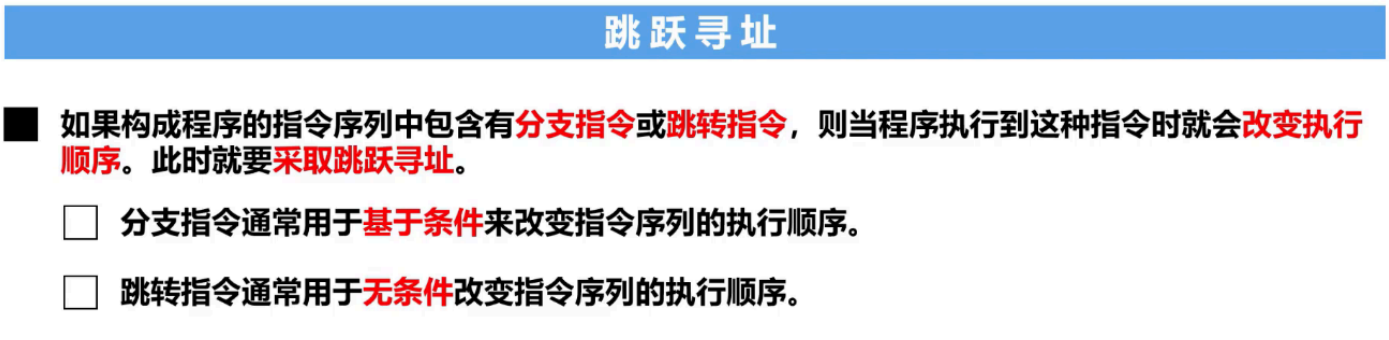
- 例1：8 位指令、按字节编址，每条指令占 1 字节，PC+1
- 例2：16 位指令、按字节编址，每条指令占 2 字节，PC+2（0x0016 → 0x0018）
- 十六进制加法按字节数加，不是按“指令条数”加

易错

- 0x0016 + 2 = 0x0018，不是 0x0017 或 0x0019
- 若指令字长 16 位但按字编址，PC 增量是 1 个字，不是 2

二、跳跃寻址

2.1 定义



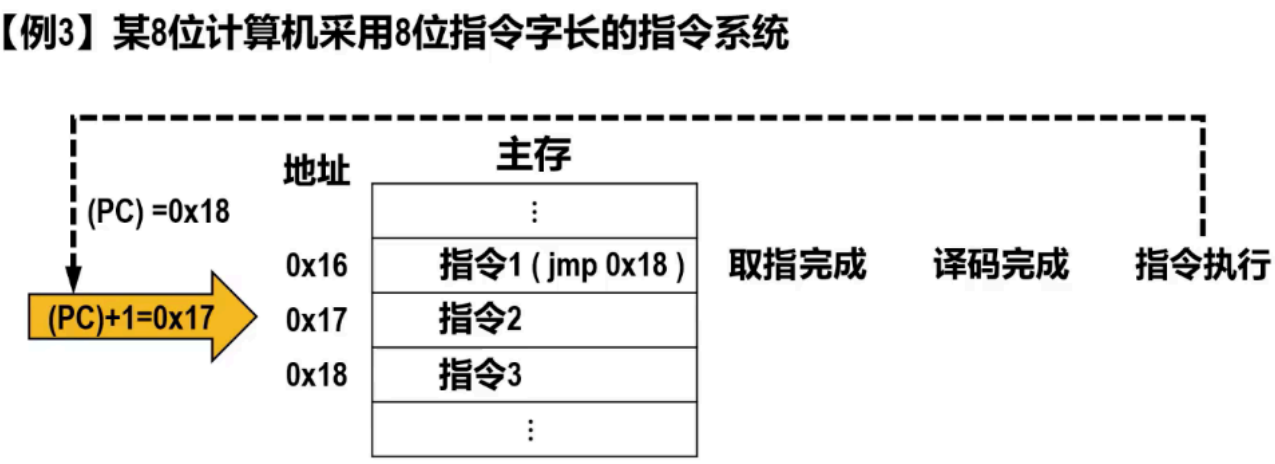
提炼

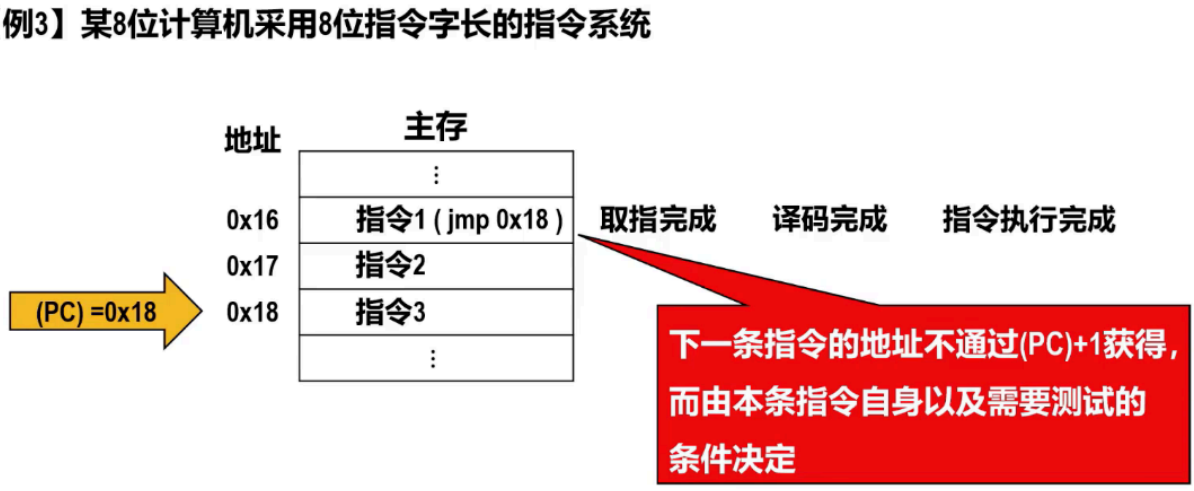
- 指令序列中出现分支/跳转时，下一条指令地址不由 PC+增量决定，而由本条指令和测试条件决定
- 分支指令：基于条件改变执行顺序；跳转指令：无条件改变执行顺序
- 转移目标通常由“更新后的 PC + 位移”或“寄存器/立即数直接给出”形成

易错

- 跳跃寻址不是“PC 不更新”；取指时 PC 仍会自增，执行阶段再用转移目标覆盖 PC
- 条件转移指令不修改标志位，只读取前面指令产生的标志

2.2 举例





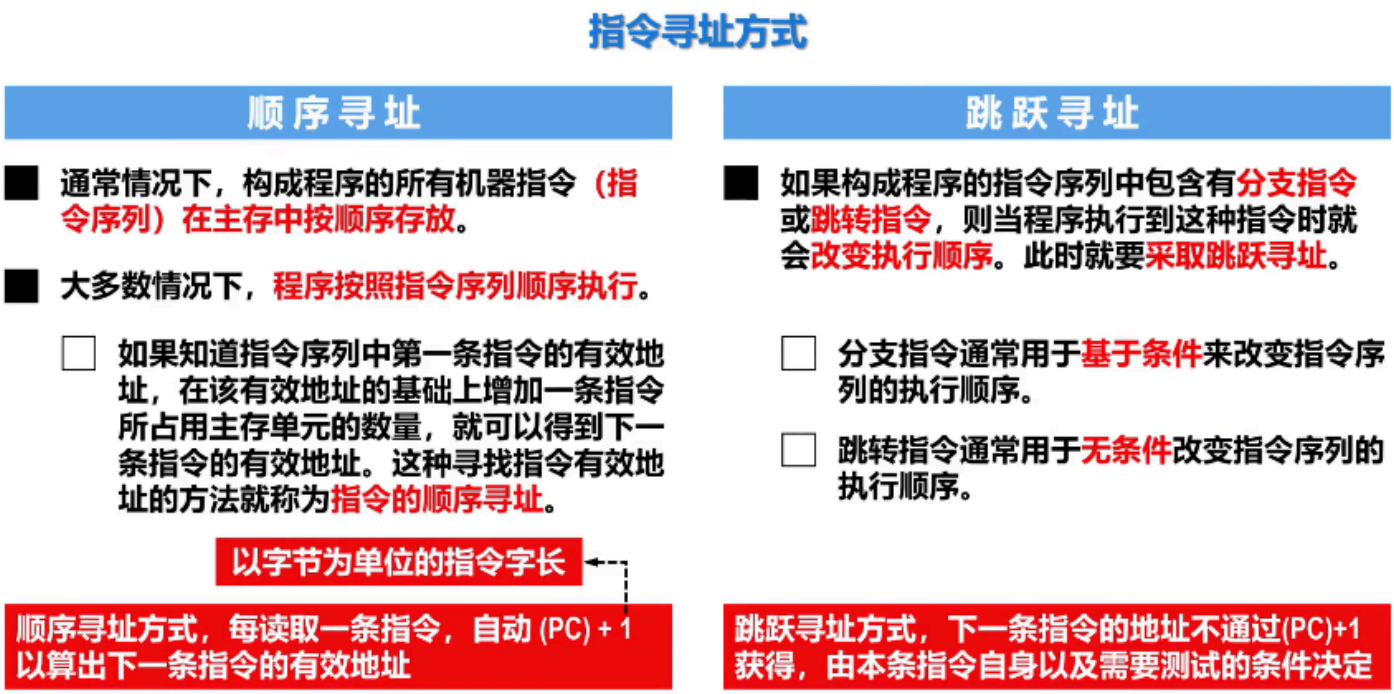
提炼

- 8 位机、8 位指令、按字节编址；`jmp 0x18` 位于 0x16
- 取指完成：PC 已由 0x16 变为 0x17
- 执行完成：`jmp 0x18` 把 PC 强行改为 0x18，下一条取指从 0x18 开始
- 关键顺序：先“取指 + PC 自增”，后“执行 + 覆盖 PC”

易错

- 若题目问转移目标地址，要看清位移是相对“取指后的 PC”还是“指令本身的地址”；408 相对寻址默认以更新后的 PC 为基准
- 转移指令本身占多个字节时，PC 自增的是整条指令长度，不是 1
- 转移目标落在指令中间（非指令边界）会导致取指错误；合法目标必须按指令长度对齐

三、总结



提炼

- 顺序寻址：PC ← PC + 指令字节数，适用于顺序执行
- 跳跃寻址：PC ← 转移目标，适用于分支、循环、调用/返回
- 两者不是互斥的：程序整体顺序执行，遇到转移指令时局部跳跃

易错

- 不要把这节的“指令寻址”和 5.6 的“操作数寻址”混为一谈；同一台机器可以指令顺序寻址、操作数变址寻址
- 子程序调用/返回也属于改变 PC，但它们还要保存/恢复返回地址，不能只按普通跳转处理

知识关联：PC 与指令字长（5.1 指令的一般格式与指令字长）；相对寻址（5.6 寻址方式——操作数寻址）；取指阶段数据流（6.2 指令执行的过程——一般过程）；分支冒险（6.29 指令流水线——流水线冲突解决（结构与控制冲突的解决））